

## Questions on Unit 05

The following multiple-choice questions are designed to assess your knowledge of this unit's section. See the version without solutions first. Each question is worth 1.0 points, with the score being divided among the correct answers where applicable. You should aim to complete each question within an average time of three to four minutes. You may solve the questions in any order you prefer; once finished, compare your responses with the provided solution file. You have successfully passed the assessment if you achieve at least 50% of the available points.

**Ex. C++ Thread Basics.**

**Choose one or more options:**

- ☐ To use `std::thread`, you must include the `<pthread.h>` header.
- ☒ The header defines the class `std::thread`, which is the standard way to start new threads.
- ☒ When compiling with `gcc` or `clang`, the `-pthread` flag is often required.
- ☐ `std::thread` can only be used with standard global functions, not lambda expressions.
- ☒ A `std::thread` object can be initialized with any callable object, such as a function or a class instance.
- ☒ The `hardware_concurrency()` function reports the maximum number of threads supported by the architecture.

**Ex. Thread Lifetime and Management.**

**Choose one or more options:**

- ☒ The `join()` member function is used to wait for a thread to finish its execution.
- ☐ `join()` must be called at least twice for every thread to ensure it is properly destroyed.
- ☒ `detach()` allows a thread to run independently from the thread object that started it.
- ☒ If an assignment is made to a thread object that is already running a function, the program terminates.
- ☐ Standard threads are copyable, allowing you to have multiple objects managing the same execution.
- ☒ `std::thread` objects are movable, which allows them to be stored in containers like `std::vector`.

**Ex. Synchronization Primitives Overview.**

**Choose one or more options:**

- ☒ C++11 introduced basic synchronization primitives like mutexes and condition variables.
- ☐ Counting semaphores were introduced as part of the C++14 standard.
- ☒ C++20 officially introduced semaphores in the headers `binary_semaphore` and `counting_semaphore`.
- ☒ A semaphore can be manually implemented using a mutex and a condition variable.
- ☐ Mutexes are primarily used to coordinate thread timing, while condition variables protect shared data.
- ☒ `std::recursive_mutex` allows a thread to acquire the same lock multiple times.

**Ex. Mutex Ownership and Behavior.**

**Choose one or more options:**

- ☒ A thread owns a mutex from the time it successfully calls either `lock` or `try_lock` until it calls `unlock`.
- ☐ It is safe for one thread to lock a mutex and a different thread to unlock it.
- ☒ The behavior of a program is undefined if a mutex is destroyed while still owned by a thread.
- ☒ `try_lock()` attempts to acquire the mutex without blocking and returns a boolean result.
- ☐ The class `std::mutex` is both copyable and movable.
- ☒ When a thread owns a mutex, all other threads attempting to lock it will block.

#### Ex. Recursive and Timed Mutexes.

##### Choose one or more options:

- ☒ `std::recursive_mutex` is useful for recursive functions that use the same mutex.
- ☒ The unlock function must be called once for each successful lock on a recursive mutex.
- ☐ A standard `std::mutex` will allow a second lock from the same thread if it is in a different function.
- ☒ `std::timed_mutex` provides the ability to attempt to claim ownership with a timeout.
- ☒ `try_lock_for` returns if the mutex remains unavailable for a specified duration.

#### Ex. Shared Mutexes.

##### Choose one or more options:

- ☒ `std::shared_mutex` (C++17) allows for multiple shared locks or one exclusive lock.
- ☒ Multiple threads can hold shared locks simultaneously.
- ☒ An exclusive lock is granted by the member functions `lock` and `unlock`.
- ☐ Shared mutexes are generally slower than standard mutexes for write-heavy scenarios.
- ☒ In POSIX, shared mutexes are referred to as Reader-Writer Locks.
- ☒ Readers typically use shared locks while writers use exclusive locks.

#### Ex. RAII Mutex Wrappers.

##### Choose one or more options:

- ☒ RAII wrappers simplify thread synchronization by managing the lock lifecycle.
- ☒ `std::lock_guard` automatically locks on creation and unlocks when the scope is exited.
- ☐ `std::lock_guard` allows for manual unlocking before destruction.
- ☒ `std::unique_lock` is more flexible and supports deferred or timed locking.
- ☒ `std::scoped_lock` (C++17) provides deadlock-free locking of multiple mutexes.
- ☐ `std::unique_lock` is copyable but not movable.

#### Ex. Advanced Unique Lock Features.

##### Choose one or more options:

- ☒ `std::defer_lock` can be used to initialize a `unique_lock` without immediately locking the mutex.
- ☒ `unique_lock` supports manual unlocking and relocking within its scope.
- ☒ `unique_lock` is required when using condition variables for waiting operations.
- ☐ `unique_lock` cannot be used with a `std::shared_mutex`.
- ☒ `unique_lock` guarantees unlocking even if an exception is thrown.

#### Ex. Deadlock Prevention.

##### Choose one or more options:

- ☒ Deadlocks can occur if different threads try to lock multiple mutexes in different orders.
- ☒ One solution to deadlock is to force a consistent locking order.
- ☐ `lock_guard` is the preferred wrapper for safely locking three mutexes at once.
- ☒ `std::scoped_lock` locks all provided mutexes using a deadlock-free algorithm.
- ☒ Scoped locks are generally very time-inefficient and should be used with care.

#### Ex. Condition Variables Purpose.

##### Choose one or more options:

- ☒ Condition variables allow threads to wait for a condition without polling or busy-waiting.
- ☒ They provide a place for threads to Rendez-vous.
- ☐ Condition variables have an internal "memory" that records signals sent before a thread started waiting.
- ☒ A mutex is always required to protect access to the shared data used with a condition variable.
- ☒ They facilitate coordination between threads based on the state of shared data.

**Ex. Condition Variable Mechanics.**

**Choose one or more options:**

- ☒ `notify_one()` wakes up a single waiting thread.
- ☒ `notify_all()` wakes up all threads currently waiting on the condition variable.
- ☒ The wait operation atomically unlocks the mutex while the thread goes to sleep.
- ☒ The mutex is automatically re-locked when the thread wakes up.
- ☐ The mutex must be held by the caller when calling `notify_one()`.
- ☒ Condition variables are defined in the header `<condition_variable>`.

**Ex. The "Wait" Pattern with Condition Variables.**

**Choose one or more options:**

- ☒ A while loop is required around the wait call to handle spurious wake-ups.
- ☐ Using an "if" statement is a safe alternative to "while" for checking the condition.
- ☒ The thread should re-evaluate the predicate whenever a condition variable returns.
- ☒ The code is broken if a signal is sent before the parent starts waiting.
- ☒ The mutex is used to protect the condition variable and must be locked before waiting.

**Ex. C++20 Semaphores.**

**Choose one or more options:**

- ☒ `counting_semaphore` allows more than one concurrent access to a resource.
- ☒ `binary_semaphore` is a specialization with a maximum value of 1.
- ☒ `acquire()` decrements the internal counter or blocks until it can.
- ☒ `release()` increments the counter and unblocks acquirers.
- ☐ A semaphore must be released by the same thread that acquired it.
- ☒ Unlike mutexes, semaphores are not tied to specific threads.

**Ex. Barriers in C++.**

**Choose one or more options:**

- ☒ Barriers coordinate multiple threads so they all continue from the same point.
- ☒ `std::latch` is a one-time synchronization primitive.
- ☒ `std::barrier` is reusable and resets after the counter reaches zero.
- ☐ A latch counter can be incremented after the latch is created.
- ☒ `arrive_and_wait()` is the standard way to synchronize at a barrier.
- ☒ Barriers generalize the thread `join()` function for an arbitrary number of threads.

**Ex. Thread Pools Concepts.**

**Choose one or more options:**

- ☒ Thread pools help achieve concurrency while reducing overhead.
- ☒ A pool maintains multiple threads waiting for tasks from a FIFO queue.
- ☐ Thread pools are directly available in the C++11 standard library.
- ☒ Thread creation and destruction are restricted to the initial and final stages of the pool.
- ☒ Over-subscription occurs when ready software threads exceed available hardware threads.
- ☒ Callback functions are often used to solve tasks within a pool.

**Ex. Atomic Operations.**

**Choose one or more options:**

- ☒ `std::atomic` provides indivisible operations on single variables.
- ☒ Atomic operations typically avoid involving the OS scheduler.
- ☒ `fetch_add()` atomically adds to a variable and returns the original value.

- ☐ Atomic variables are both copyable and movable.
- ☒ `load()` and `store()` are used to retrieve and replace atomic values.

#### Ex. Task-Based Programming.

##### Choose one or more options:

- ☒ `std::async` is the primary tool for task-based parallel programming.
- ☒ Unlike `std::thread`, tasks have the possibility of handling return values.
- ☒ `launch::async` forces a new thread to be generated.
- ☒ `launch::deferred` allows the OS to run the function only when results are requested.
- ☐ The default policy for `std::async` is always `launch::async`.

#### Ex. Futures and Promises.

##### Choose one or more options:

- ☒ A promise is an object that stores a value to be retrieved by a future.
- ☒ `future::get()` blocks the thread until the object is ready.
- ☒ `std::shared_future` can be copied and accessed by multiple instances.
- ☐ `get()` can be called multiple times on a unique `std::future` object.
- ☒ A single promise-future pair is designed to transmit one value exactly once.
- ☒ `set_value()` more than once on the same promise results in an exception.

#### Ex. Task Management and Policies.

##### Choose one or more options:

- ☒ `wait_for()` can be used to check the status of a future with a timeout.
- ☒ If a policy is deferred, `wait_for()` returns `future_status::deferred`.
- ☒ The system balances tasks automatically through work-stealing features.
- ☐ Tasks are associating with threads in a manual way by the programmer.
- ☒ futures can catch exceptions thrown by the task function.

#### Ex. Advanced Concurrency Strategies.

##### Choose one or more options:

- ☒ Semaphore throttles limit the number of active workers in expensive code sections.
- ☒ Atomic operations are limited in scope but faster than mutexes.
- ☒ Thread-based approaches are useful for long-running background tasks.
- ☐ Promises and futures are the right tool for communicating multiple results over time.
- ☒ `std::move` is used to transfer a promise into a task.